ARTIFICIAL NEURAL NETWORK

[ANN]

INTRODUCTION

- In this assignment, we implement a fully connected neural feed feedforward neural network from multiple in build function and librarys of python .The objective is to train the model using **backpropagation and gradient decent based optimization algorithms** for multiclass classification on the FASHION-MNIST dataset.

- The dataset consist of **28*28 (784 columns and 55000 rows) grayscale images** belongs to **10 deffrent categories**. Various **hyperparameters** such as number of **hidden layers, hidden layer size , learning rate batch size and activation functions and loss functions and optimization algorithms** are explored

- The **goal of this study is to analyze the effect of architectural and optimization choice on training and validation performance**.

DATASET DESCRIPTION

- The experiments in this assignment are conducted using the Fashion-MNIST dataset. Fashion MNIST is publicly available benchmark dataset introduced by Zalando Research as a more challenging replacement for traditional handwritten digit dataset

- In This Dataset Have Consist :-

1. Total Training Sample(rows) :- **60000**
2. Test Samples :- **10000**
3. Image Size :- **28X28 Pixels**
4. Input Dimension :- **784(flattened Image)**
5. Number Of Classes :- **10(0-9)**
6. Image Type :- **Grayscale**

- **Here 0-9 lables represent a input of fashion MNIST dataset**
- 0. ----- t-shirt/top
- 1.------- trouser

- 2. ------- Pullover
- 3.--------- Dress
- 4.-------- Coat
- 5.--------- Sandal
- 6.--------- shirt
- 7.----------Sneaker
- 8.---------Bag
- 9.--------- Ankle boot

❑ INPUT REPRESENTATION

Each 28X28 grayscale image is flattened into 784-dimensional vector before being fed into neural network.

Pixels values range from 0 to 255.

To improve training stability, the inputs are Normalized as:

$$x = x/255$$

This scales pixels values to the range [0-1].

And a machine work better on scale values.

❑ OUTPUT REPRESENTATION

- ✓ This is a multiclass classification problem, the output layer contains 10 neurons.
- ✓ Labels are converted into one-hot-encoded vectors.

- ✓ Example :-

  label 4 ---→ [0,0,0,0,1,0,0,0,0,0]

❑ DATASET SPLIT

- ✓ The training dataset is further divided into

  - Training set (available on kaggle)
  - Validation set(available on kaggle)

- ○ Validation loss is used for hyperparameter tuning .

NETWORK ARCHITECTURE

- The neural network implemented in this assignment is a fully connected feedforward neural network trained using backpropagation.

❖ STRUCTURE
  - Input layers :- 784 neurons
  - Output layers :- 10 neurons
  - Hidden layers :- 1-4(based on experiment instruction)
  - Activation functions :- Sigmoid/Tanh
  - Output Activation : Softmax

- **MATHEMETICAL FORMULAS**
  - **Z = W(transpose).X + b**
  - **A = f(z)**

W = weights

B = bias

F = activation functions

❑ ACTIVATION FUNCTIONS

→ SIGMOID :-

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid is a very popular activation functions , it also called as probabilistic activation function(logistic function).

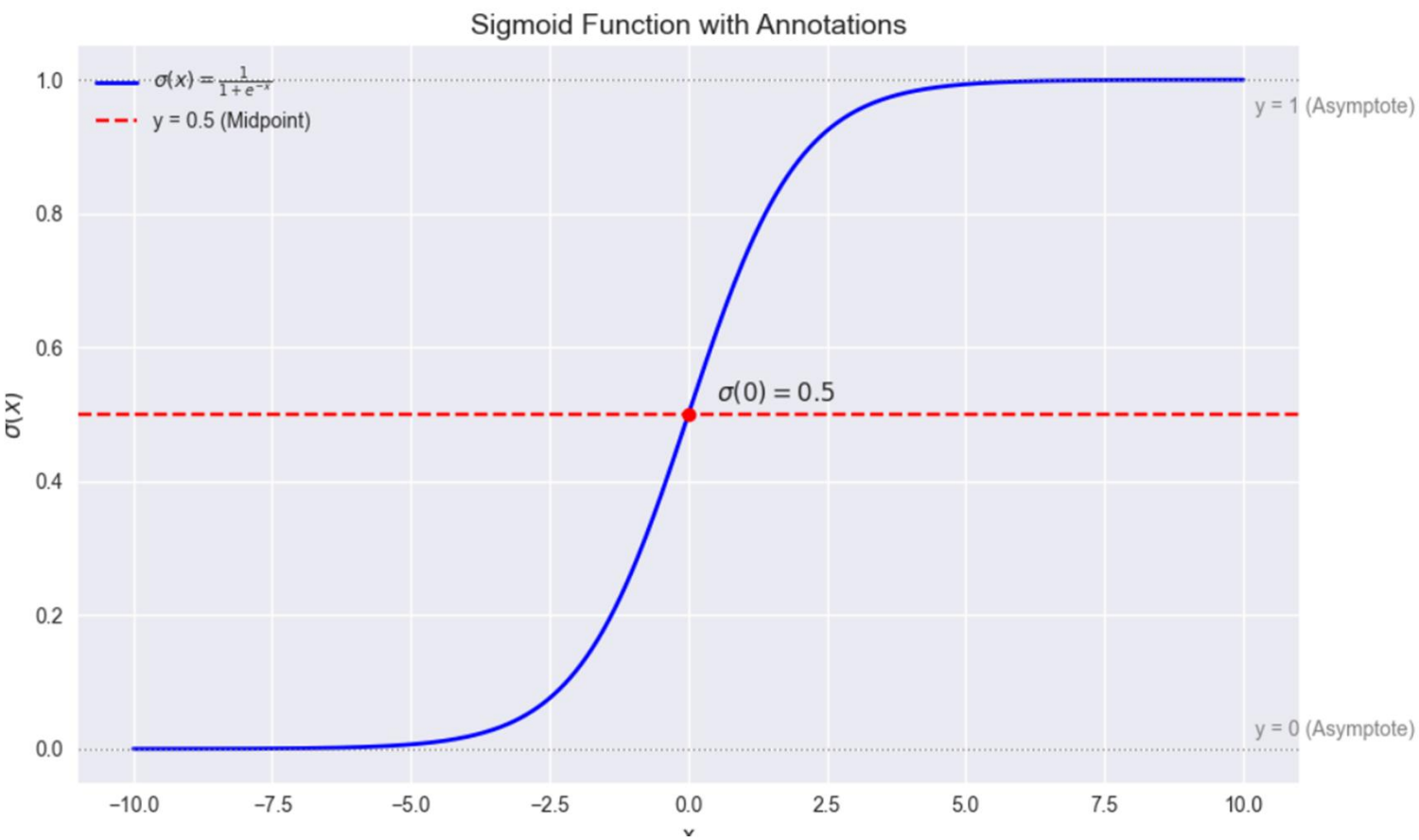
Which is return output in range from 0 to 1.

Sigmoid function graph is looking like shape of S , the top edge of S curve is known as 1 in sigmoid----- and bottom edge of sigmoid known as 0

This is very use full in optimization and backpropagation in neural networks.

It is most use for binary class classification.

Here I provide graph which is explain how the sigmoid activation function works on clasification problems .



- This is sigmoid activation function graph . Which is represent decision boundary between S shape in red line .
- If X = 0 than boundary line is 0.5
- If value of output>0.5 than → answer is 1
- If value of output<0.5 than → answer is 0

❑ → Tanh (hyperbolic tangent)..

- **Hyperbolic tangent means Tanh is a also activation function which is used in neural network most.**
- **The sigmoid is return output in range(0-1) but Tanh is return output in range of like scaling values (-1 to 1).**
- **Sigmoid have some problems like if is transform minus values in positive form**
- **Means negative values transform in positive values but it is some time loss data point importance .**

- **Importance of Tanh is , it return output in negative and positive form on based on output , means is output is a big positive number than Tanh return output in side of +1-------- if value of input is small positive or negative than Tanh is return values in side of -1.**

- **This output also based on of formulas which is work in back of Tanh , which decided output based on input which is it required for calculation.**

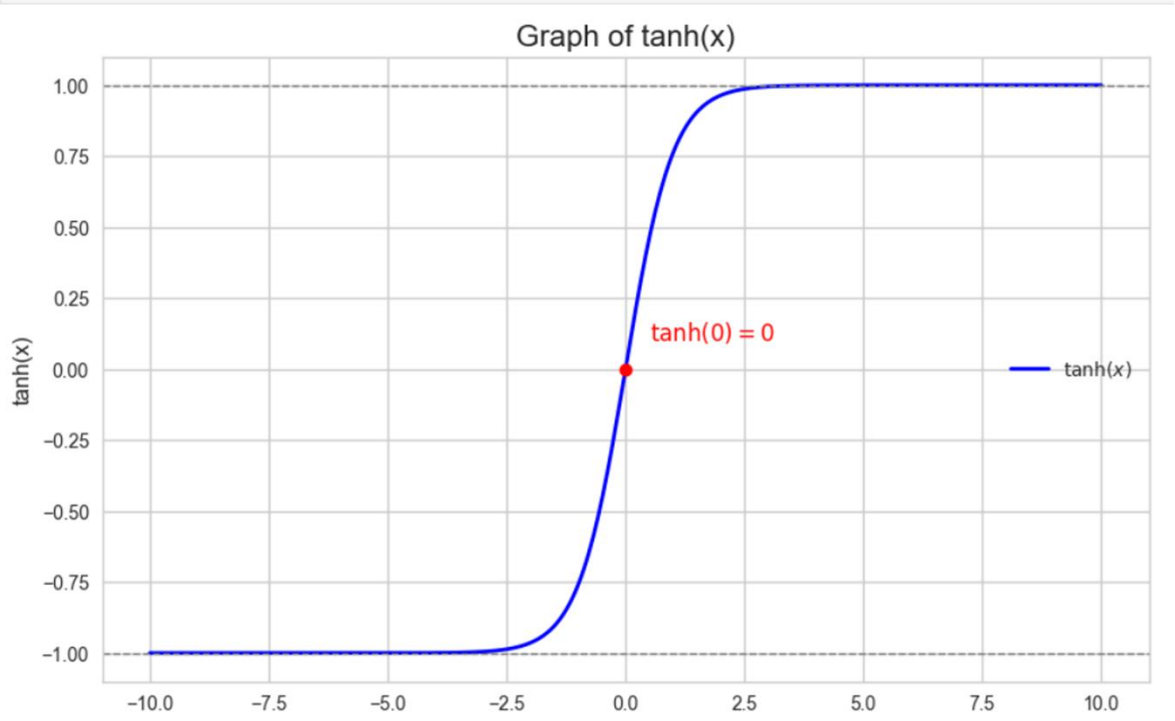
- **Now we see graph intution of tanh and try to understand how work it on input.**

- **Formula of …Tanh….:-**

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **This formula is used exponantial function which is transform input values in 0-1 range mostaly.**

- **Now we see graph of Tanh which is looking as sigmoid but different in range .**



LOSS FUNCTIONS

- If we see our title ---**gradient descent (and its variants) with backpropagation**-----**gradient decent with backpropagation is use for minimize the loss to change values of omega(w) and bias(b).**

- **But question is that how to we calculate losses in neural network , then we know ki we need to use backpropagation with GD.**
- **So there are some popular function in neural network or deep learning**

  - **CROSS ENTROPY LOSS--------→**
  - **SQUARED ERROR LOSS--------→**

- **These two are most use for calculate loss of training data and validation data**
- **I have allready use for calculate loss using cross entropy and MSE and I See , a Cross entropy is better than MSE.**

❑ CROSS ENTROPY V/S MEAN SQUARED ERROR

MSE :-

$$L = \frac{1}{2}(y - \hat{y})^2$$

- When we use sigmoid + MSE then derivative is transform In small form in output layers
- With this combination a gradient is also transform in small form
- And a up dation of weightage is run slow
- And last is training perform will be very slow

- Its all called as Vanishing gradient problem.

Cross entropy :-

$$L = -\sum y \log(\hat{y})$$

- Cross entropy loss is better than MSE loss function because it give better convergence comparing with sigmoid
- And a combination of cross entropy and softmax is very better combination for multiclass classification

- In cross entropy and sigmoid:

✓ Gradient will be simplify

✓ Output error(y_pred-Y_actual)

✓ Gradient vanish problem **X**

✓ Strong weights update

- And it gives us better convergence in few epochs and gives better minimum loss.

- Cross entropy loss generally provides better convergence compared to squared error loss in classification tasks. When used with soft max activation, the gradient simplifies and avoids the vanishing gradient problem. This results is fastest and more stable learning.
- In contrast , squared error loss may produced smaller gradients, slowing down training and leading to suboptimal convergence.

WHY COMPARE OPTIMIZERS

- ✓ When we compare different optimizers than result is that…….
  - Convergance speed
  - Training stability
  - Validation accuracy
  - Generalization performance
- ✓ Optimizers :…..
  i. SGD+Momentum
  ii. NAG
  iii. ADAM

These are optimizers which is used in this ANN projects to calculate loss minimize loss get better accuracy of the model….

WORKING MECHANISM COMPARISION

OPTIMIZERS	HOW TO WORK	SPECIAL FEATURE
MOMENTUM	Add previous gradient	Smooth direction
NAG	Respect to future position gradient	Look-ahead update
ADAM	Store Mean + variance	Adaptive learning rate

- KEY DIFFRENCE
  i. MOMENTUM past dekhta he
  ii. NAG future dekhta he
  iii. ADAM smart adaptive learning rate use krta he.

OPTIMIZATION ALGORITHMS

❑ GRADIENT DESCENT ALGORITHMS

- Gradient descent algorithms is a loss minimization technique which is use for update values of omega and bias for minimize the loss and plot best graph for presentation and work best with model.

- Gradient descent :-
  - **Gradient means :- which direction he have loss increse**
  - **Decscent means : run opposite direction of gradient**

- Formula of gradient descent :-→

$$W = W - \eta \nabla L \longrightarrow$$ Terms of omega update

$$b_{\text{new}} = b_{\text{old}} - \eta \cdot \frac{\partial L}{\partial b} \longrightarrow$$ Terms of bias up update

❑SGD-stochastic gradient descent

- Stochastic gradient descent is an algorithms which is update parameters to minimize loss function.
- It's a different from traditional gradient descent , because its update parameters on sample data set not entire dataset .

- **Formula :-**
  - **W = W – lr * derivative(L(W)) → standard gradient descent**
  - **SGD update rule = W = W – lr * derivative(L(W);Xi,Yi)**
    - **Xi = single training sample**
    - **Yi = corresponing label**
    - **Learning rate = lr**
- **Working mechanism**
  1. **Choose sample randomly**
  2. **Forward pass**
  3. **Loss calculate**
  4. **Backpropagation and GD**
  5. **Weight updation**
  6. **Work with next sample**

METHAMETICAL COMPARISION

Momentum

$$V = \beta V + \eta \nabla L$$

$$W = W - V$$

NAG

$$V = \beta V + \eta \nabla L(W - \beta V)$$

Adam

$$m = \beta_1 m + (1 - \beta_1) \nabla L$$

$$v = \beta_2 v + (1 - \beta_2)(\nabla L)^2$$

$$W = W - \frac{\eta}{\sqrt{v} + \epsilon} m$$

CONVERGENCE SPEED COMPARISION

OPTIMIZERS	CONVERGENCE SPEED
MOMENTUM	Fast
NAG	Faster
ADAM	Fastest

STABILITY AND ACCURACY COMPARISION

OPTIMIZERS	STABILITY	VALIDATION ACCURACY
MOMENTUM	Medium	Good
NAG	High	Better
ADAM	Very high	Best

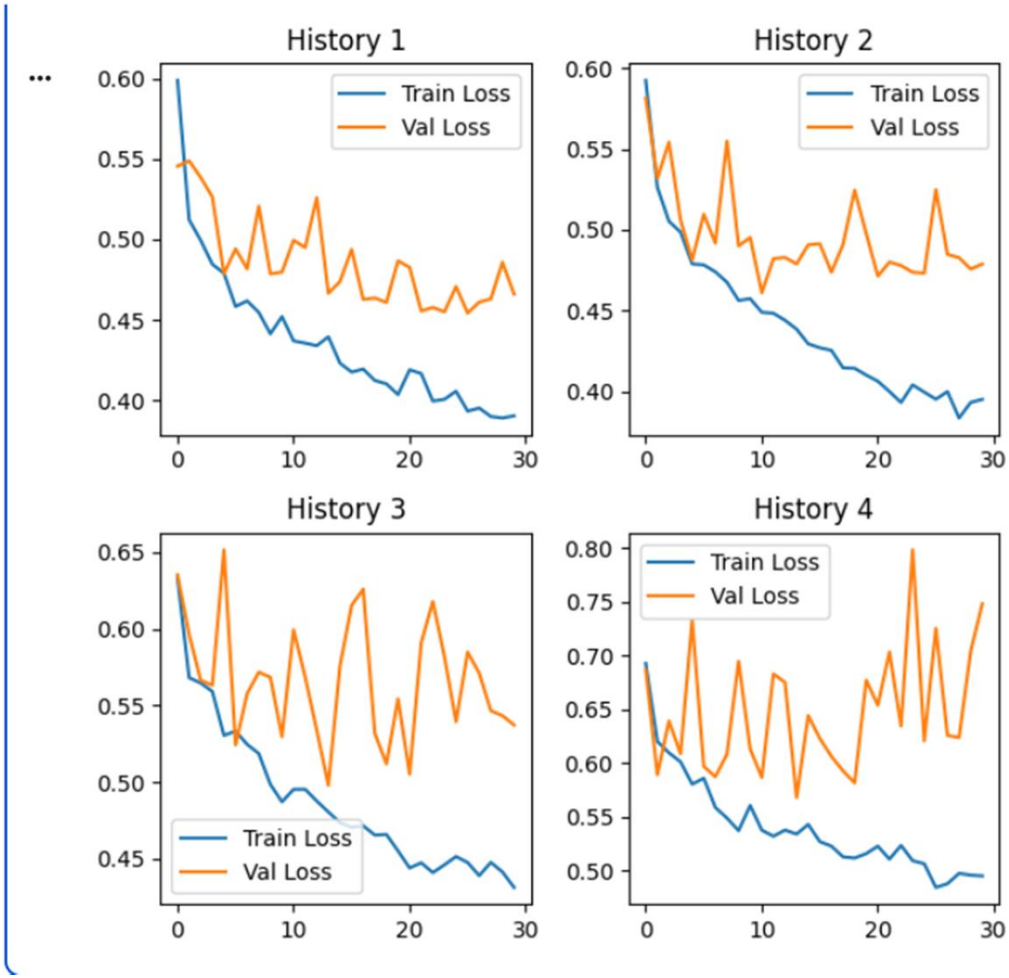
CONCLUSION OF OPTIMIZERS

- Among all optimizers, Adam showed the fastest convergence and highest validation accuracy
- NAG performed better than standard momentum due to look ahead gradient calculation.
- Momentum improved over basic SGD but was slower compared to Adam

TRAINING PIPELINE

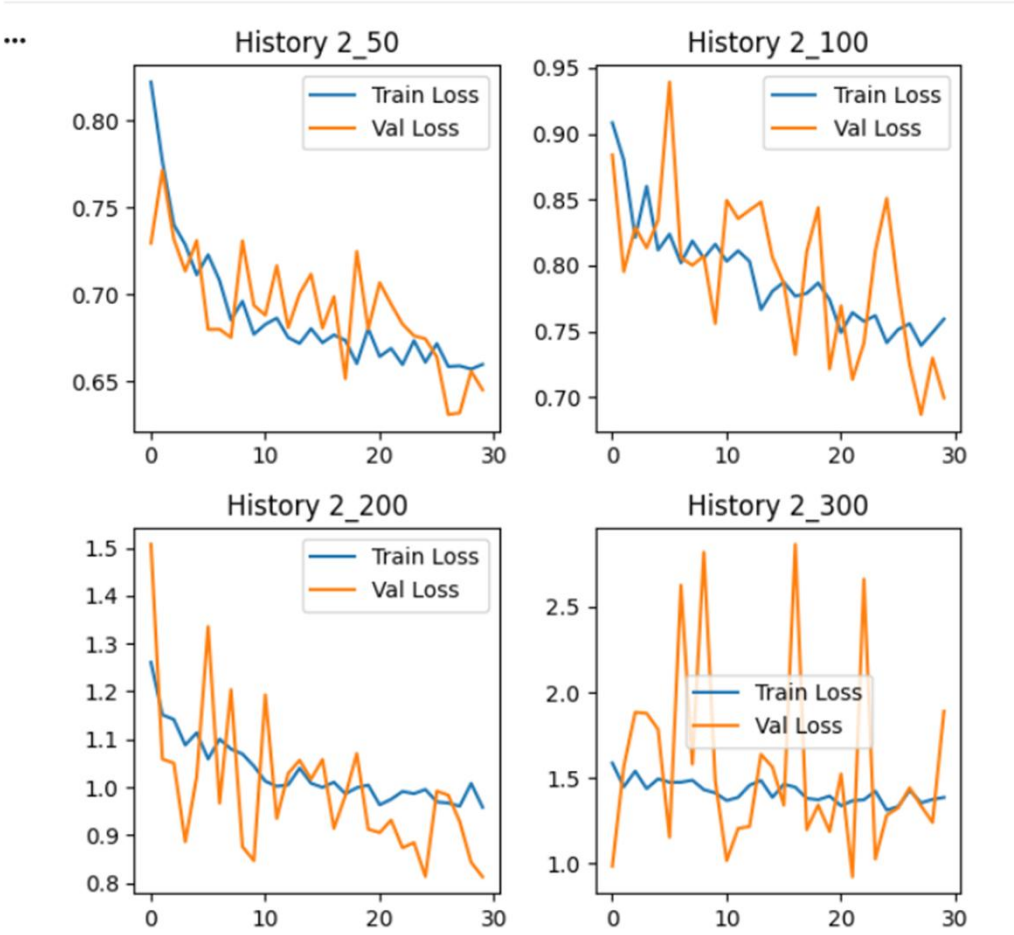
- **The neural network model is implemented using high-level deep learning libraries available in python. Built-in modules are used for defining layers, activation functions, loss computation and optimization algorithms.**
  - Data preproccesing
  - Model building
  - Compilation step
  - Training procedure
  - Logging
- **While the theoretical formulation of backpropagation and gradient descent discussed in this report , the experimental implementation leverages built-in-automatic differentiation and optimization routines provided by the deep learning frame work.**

Experiment 1
(Effect of Hidden Layer Size (1 Hidden Layer))



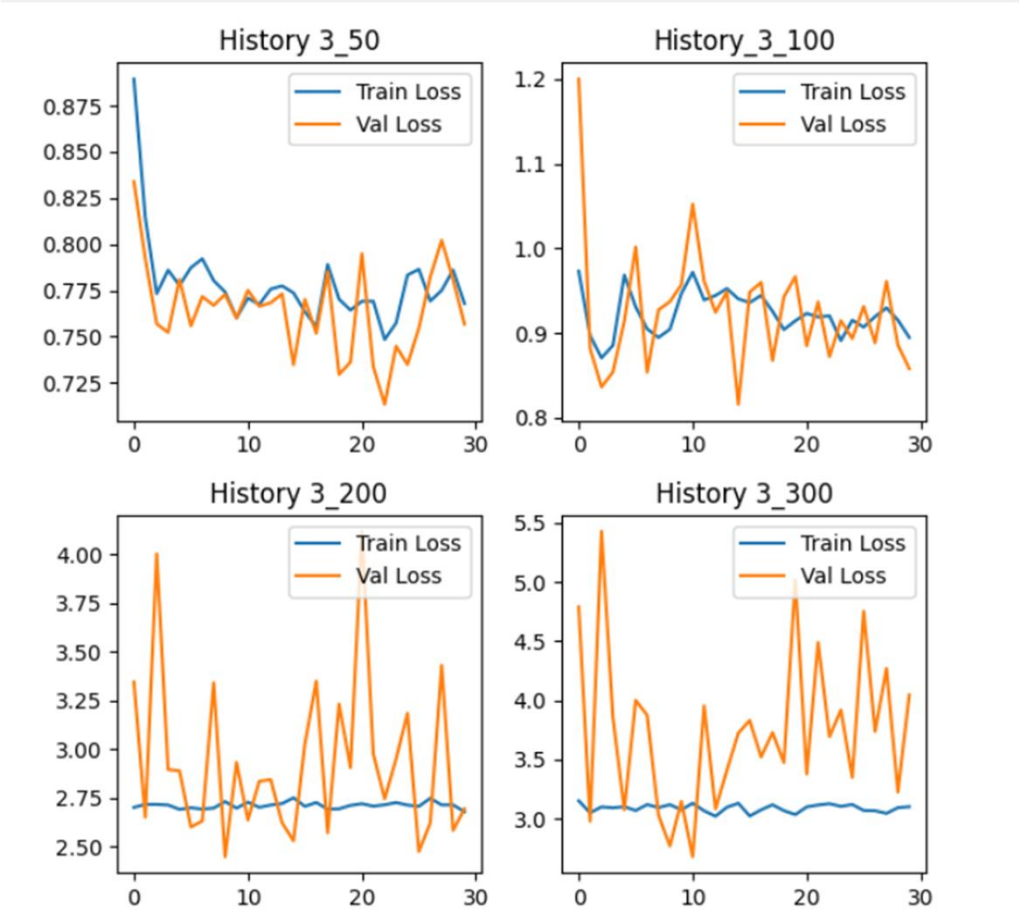
As the hidden layer size increase, the model capacity improves, resulting in faster convergence and lower training loss. Models with 50 nwurons show slower convergence and slightly higher loss, including possible underfitting. the coniguration with 100 neurons achieves the lowest training loss, demonstating better representation capabililty, and other are not working good.

Experiment 2



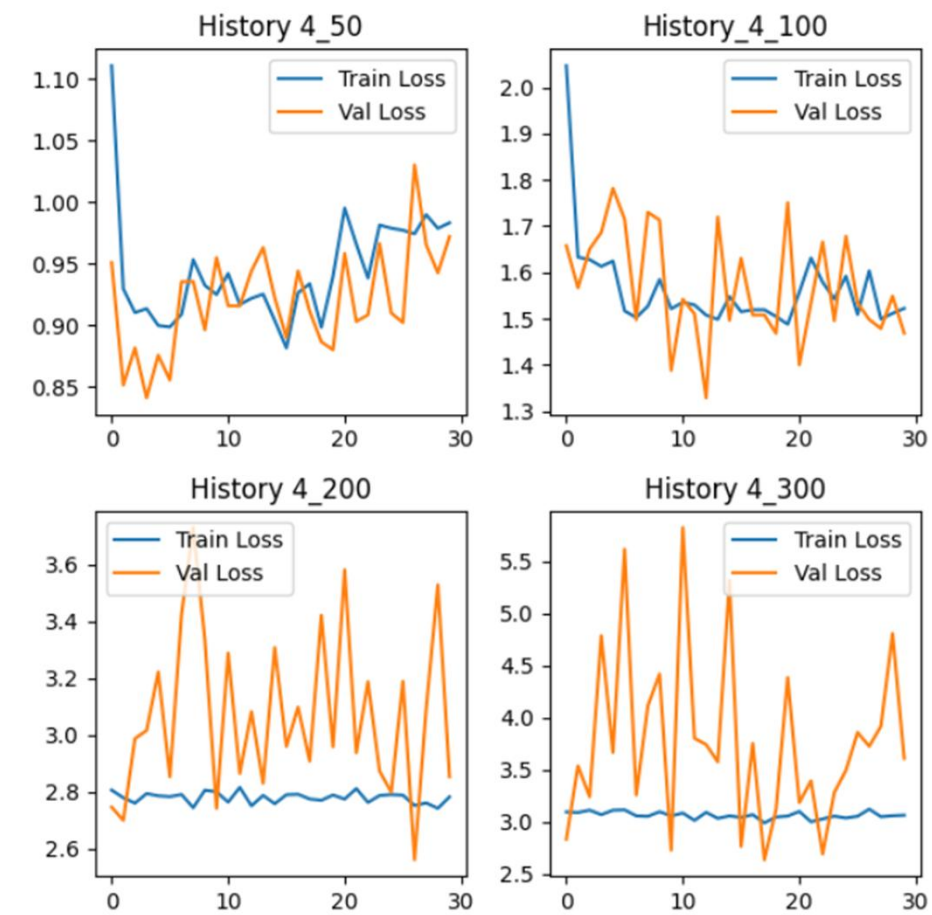
As the hidden layer size increase, the model capacity improves, resulting in faster convergence and lower training loss. Models with 50 neurons with two hidden layers faster convergence and slightly lower loss. the configuration with 300 with 2 hidden layers neurons achieves the highest training loss, demonstrating worst representation capability, and other are working good.

Experiment 3



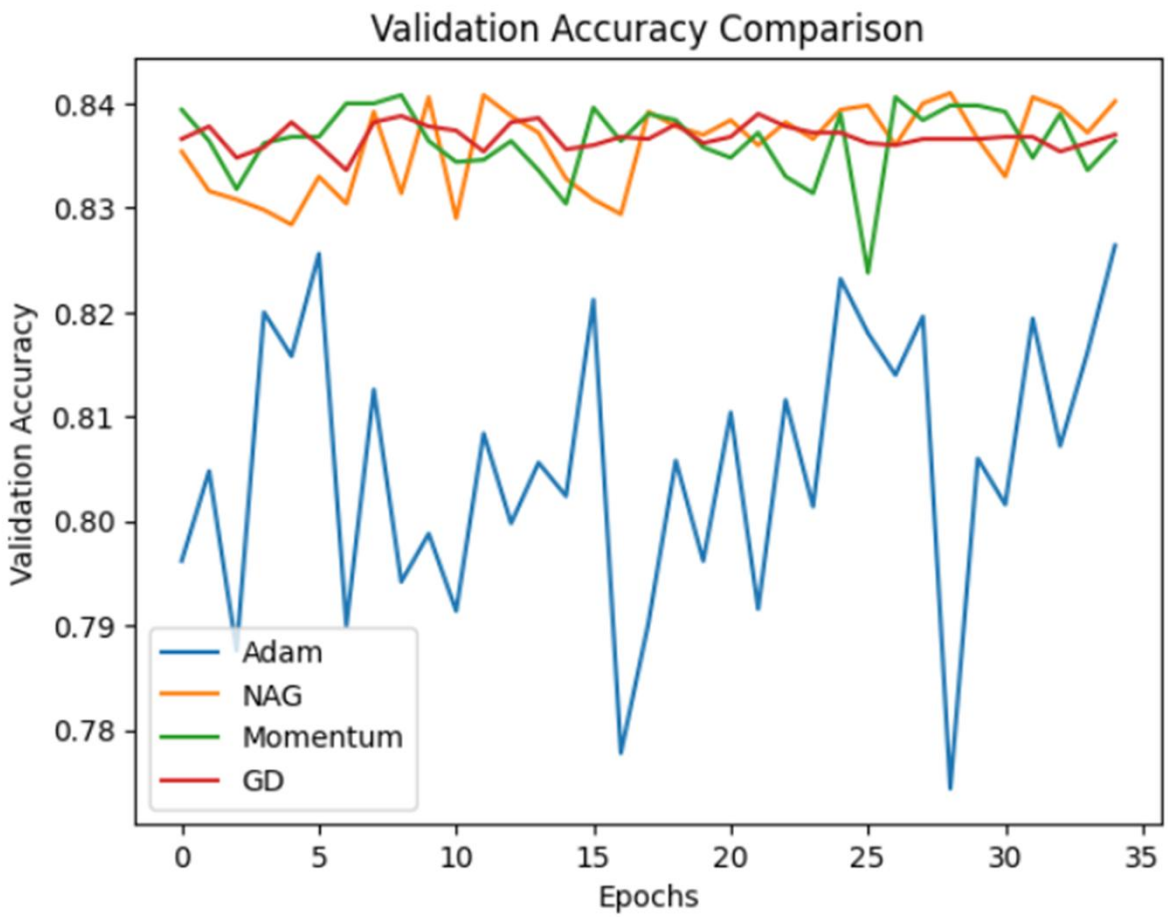
As the hidden layer size increase, the model capacity improves, resulting in faster convergence and lower training loss. Models with 50 neurons with 3 hidden layers faster convergence and slightly lower loss. the configuration with 300 with 3 hidden layers neurons achieves the highest training loss, demonstrating worst representation capability, and other are working right.

Experiment 4



As the hidden layer size increase, the model capacity improves, resulting in faster convergence and lower training loss. Models with 50 neurons with 4 hidden layers faster convergence and slightly lower loss. the configuration with 300 with 4 hidden layers neurons achieves the highest training loss, demonstrating worst representation capability, and other are working right.

Experiment 5
(optimizer comparision)



- The graph compares validation accuracy across four optimizers: Adam, NAG, Momentum, and Gradient Descent.
  Adam shows higher fluctuations, indicating instability despite its adaptive learning capability.
  NAG and Momentum maintain steadier accuracy, balancing convergence speed with generalization.
  Gradient Descent stands out as the most consistent, holding accuracy near 0.835 throughout training.
  This stability highlights GD's reliability, even if it converges slower than adaptive methods.
  Overall, GD provides the most predictable validation accuracy, while Adam remains less stable.

FINAL CONCLUSION

- ✓ **In this assignment, we implemented a feed forward neural network from using in build librarys of python using backpropagation to evaluated different hyperparameters on the fashion-MNIST dataset.**

- ❖ **We analyzed the effect :-**

  - **Number of hidden layers**
  - **Hidden layer size**
  - **Activation functions (sigmoid/tanh)**
  - **Loss functions (cross entropy vs squared error)**
  - **Optimizers (GD, momentum, NAG, Adam)**
  - **Batch size variation**

✓ From the experiments, the following observation were made:

1. Increasing hidden layer size improved learning capacity but very large sizes increased overfitting.
2. Cross-entropy loss performed better than squared error for classification tasks.
3. Sigmoid and tanh both worked, but tanh showed slightly better gradient flow.
4. Among optimization algorithms, Adam achieved the fastest convergence and lowest validation loss followed by the NAG, Momentum, and GD.
5. Moderate batch size provided a good balanced beetween stability and speed.

✓ **Therefore, the best overall performance was obtained using:**

- **Adam optimizers**
- **Cross entropy loss**
- **Appropriate learning rate tuning**
- **2-3 hidden layers with sufficient neurons**

THANK YOU AND

HAPPY LEARNING